- **Decomposing Activation Space.** The most fundamental challenge of any interpretability agenda is to defeat the curse of dimensionality. For mechanistic interpretability, this ultimately reduces to whether we can decompose activation space into independently understandable components, analogous to how computer program memory can be decomposed into variables. Identifying features is what allows us to decompose the model in terms of them.
- **Describing Activations in Terms of Pure Features.** One of the most obvious casualties of superposition is that we can't describe activations in terms of pure features. When features are relatively basis aligned, we can take an activation – say the activations for a dog head in a vision model – and decompose them into individual underlying features, like a floppy ear, short golden fur, and a snout. (See the "semantic dictionary" interface in Building Blocks [37] .) Solving superposition would allow us to do this for every model.
- **Understanding Weights (ie. Circuit Analysis).** Neural network weights can typically only be understood when they're connecting together understandable features. All the circuit analysis seen in the original circuit thread (see especially [38]), see specially was fundamentally only possible because the weights connected non-polysemantic neurons. We need to solve superposition for this to work in general.
- **Even very basic approaches become perilous with superposition.** It isn't just sophisticated approaches to interpretability which are harmed by superposition. Even very basic methods one might consider become unreliable. For example, if one is concerned about language models exhibiting manipulative behavior, one might ask if an input has a significant cosine similarity to the representations of other examples of deceptive behavior. Unfortunately, superposition means that cosine similarity has the potential to be misleading, since unrelated features start to be embedded with positive dot products to each other. However, if we solve superposition, this won't be an issue – either we'll have a model where features align with neurons, or a way to use compressed sensing to lift features to a space where they no longer have positive dot products.

Three Ways Out

At a very high level, there seem to be three potential approaches to resolving superposition:

- **Create models without superposition.**
- **Find an overcomplete basis** that describes how features are represented in models with superposition.
- **Hybrid approaches** in which one changes models, not resolving superposition, but making it easier for a second stage of analysis to find an overcomplete basis that describes it.

Our sense is that all of these approaches are possible if one doesn't care about having a competitive model. For example, we believe it's possible to accomplish any of these for the toy models described in this paper. However, as one starts to consider serious neural networks, let alone modern large language models, all of these approaches begin to look very difficult. We'll outline the challenges we see for each approach in the following sections.

With that said, it's worth highlighting one bright spot before we focus on the challenges. You might have believed that superposition was something you could never fully get rid of, but that doesn't seem to be the case. All our results seem to suggest that superposition and polysemanticity are phases with sharp transitions. That is, there may exist a regime for every model where it has no superposition or polysemanticity. The question is largely whether the cost of getting rid of or otherwise resolving superposition is too high.

APPROACH 1: CREATING MODELS WITHOUT SUPERPOSITION

It's actually quite easy to get rid of superposition in the toy models described in this paper, albeit at the cost of a higher loss. Simply apply at L1 regularization term to the hidden layer activations (i.e. add $\lambda||h||_1$ to the loss). This actually has a nice interpretation in terms of killing features below a certain importance threshold, especially if they're not basis aligned. Generalizing this to real neural networks isn't trivial, but we expect it can be done.

However, it seems likely that models are significantly benefitting from superposition. Roughly, the sparser features are, the more features can be squeezed in per neuron. And many features in language models seem very sparse! For example, language models know about individuals with only modest public presences, such as several of the authors of this paper. Presumably we only occur with frequency significantly less than one in a million tokens. As a result, it may be the case that superposition effectively makes models much bigger.

All of this paints a picture where getting rid of superposition may be fairly achievable, but doing so will have a large performance cost. For a model with a fixed number of neurons, superposition helps – potentially a lot.

But this is only true if the constraint is thought of in terms of neurons. That is, a superposition model with n neurons likely has the same performance as a significantly larger monosemantic model with kn neurons. But neurons aren't the fundamental constraint: flops are. In the most common model architectures, flops and neurons have a strict correspondence, but this doesn't have to be the case and it's much less clear that superposition is optimal in the broader space of possibilities.

One family of models which change the flop-neuron relationship are Mixture of Experts (MoE) models (*see review* [39]). The intuition is that most neurons are for specialized circumstances and don't need to activate most of the time. For example, German-specific neurons don't need to activate on French text. Harry Potter neurons don't need to activate on scientific papers. So MoE models organize neurons into blocks or experts, which only activate a small fraction of the time. This effectively allows the model to have k times more neurons for a similar flop budget, given the constraint that only $1/k$ of the neurons activate in a given example and that they must activate in a block. Put another way, MoE models can recover neuron sparsity as free flops, as long as the sparsity is organized in certain ways.

It's unclear how far this can be pushed, especially given difficult engineering constraints. But there's an obvious lower bound, which is likely too optimistic but is interesting to think about: what if models only expended flops on neuron activations, and recovered the compute of all non-activating neurons? In this world, it seems unlikely that superposition would be optimal: you could always split a polysemantic neuron into dedicated neurons for each feature with the same cost, except for the cases where there would have been interference that hurt the model anyways. Our preliminary investigations comparing various types of superposition in terms of "loss reduction per activation frequency" seem to suggest that superposition is not optimal on these terms, although it asymptotically becomes as good as dedicated feature dimensions. Another way to think of this is that superposition exploits a gap between the sparsity of neurons and the sparsity of the underlying features; MoE eats that same gap, and so we should expect MoE models to have less superposition.

To be clear, MoE models are already well studied, and we don't think this changes the capabilities case for them. (If anything, superposition offers a theory for why MoE models have not proven more effective for capabilities when the case for them seems so initially compelling!) But if one's goal is to create competitive models that don't have superposition, MoE models become interesting to think about. We don't necessarily think that they specifically are the right path forward – our goal here has been to use them as an example of why we think it remains plausible there may be ways to build competitive superposition-free models.

APPROACH 2: FINDING AN OVERCOMPLETE BASIS

The opposite strategy of creating a superposition-free model is to take a regular model, which has superposition, and find an overcomplete basis describing how features are embedded after the fact.

This appears to be a relatively standard *sparse coding* problem, where we want to take the activations of neural network layers and find out which directions correspond to features.[20]

The advantage of this is that we don't need to worry about whether we're damaging model performance. On the other hand, many other things are harder:

- **It's no longer easy to know how many features you have to enumerate.** A monosemantic model represents a feature per neuron, but when finding an overcomplete basis there's an additional challenge of identifying how many features to use for it.

- **Solutions are no longer integrated into the surface computational structure.** Neural networks can be understood in terms of their surface structure – neurons, attention heads, etc – and virtual structure that implicitly emerge (*e.g.* virtual attention heads [40]). A model described by an overcomplete basis has "virtual neurons": there's a further gap between the surface and virtual structure.

- **It's a different, major engineering challenge.** Seriously attempting to solve superposition by applying sparse coding to real neural nets suggests a *massive* sparse coding problem. For truly large language models, one would be starting with something like a millions (neurons) by billions (tokens) matrix and then trying to do an extremely overcomplete factorization, perhaps trying to factor it to be a thousand or more times *larger*. This is a major engineering challenge which is different from the standard distributed training challenges ML labs are set up for.

- **Interference is no longer pushing in your favor.** If you try to train models without superposition, interference between features is pushing the training process to have less superposition. If you instead try to decode superposition after the fact, whatever amount of superposition is "baked in" by the training process and you don't have part of the objective pushing in your favor.

APPROACH 3: HYBRID APPROACHES

In addition to approaches which address superposition purely at training time, or purely after the fact, it may be possible to take "hybrid approaches" which do a mixture. For example, even if one can't change models without superposition, it may be possible to produce models with *less* superposition, which are then easier to decode.[21] Alternatively, it may be possible for architecture changes to make finding an overcomplete basis easier or more computationally tractable in large models, separately from trying to reduce superposition.

Additional Considerations

Phase Changes as Cause For Hope. Is totally getting rid of superposition a realistic hope? One could easily imagine a world where it can only be asymptotically reduced, and never fully eliminated. While the results in this paper seem to suggest that superposition is hard to get rid of because it's actually very useful, the upshot of it corresponding to a phase change is that there's a regime *where it totally doesn't exist*. If we can find a way to push models in the non-superposition regime, it seems likely it can be totally eliminated.

Any superposition-free model would be a powerful tool for research. We believe that most of the research risk is in whether one can make *performant* superposition free models, rather than whether it's possible to make superposition free models at all. Of course, ultimately, we need to make performant models. But a non-performant superposition free model could still be a very useful research tool for studying superposition in normal models. At present, it's challenging to study superposition in models because we have no ground truth for what the features are. (This is also the reason why the toy models described in this paper can be studied – we do know what the features are!) If we had a superposition-free model, we may be able to use it as a ground truth to study superposition in regular models.

Local bases are not enough. Earlier, when we considered the geometry of non-uniform superposition, we observed that models often form *local orthogonal bases*, where co-occurring features are orthogonal. This suggests a strategy for locally understanding models on sufficiently narrow sub-distributions. However, if our goal is to eventually make useful statements about the safety of models, we need mechanistic accounts that hold for the full distribution (and off distribution). Local bases seem unlikely to give this to us.

Discussion

To What Extent Does Superposition Exist in Real Models?

Why are we interested in toy models? We believe they are useful proxies for studying the superposition we suspect might exist in real neural networks. But how can we know if they're actually a useful toy model? Our best validation is whether their predictions are consistent with empirical observations regarding polysemanticity. To the best of our knowledge they are. In particular:

- **Polysemantic neurons exist.** Polysemantic neurons form in our third model, just as they are observed in a wide range of neural networks.
- **Neurons are sometimes "cleanly interpretable" and sometimes "polysemantic", often in the same layer.** Our third model exhibits both polyemantic and non-polysemantic neurons, often at the same time. This is analogous to how real neural networks often have a mixture of polysemantic and non-polysemantic neurons in the same layer.
- **InceptionV1 has more polysemantic neurons in later layers.** Empirically, the fraction of neurons which are polysemantic in InceptionV1 increases with depth. One natural explanation is that as features become higher-level the stimuli they detect become rarer and thus sparser (for example, in vision, a high-level floppy ear feature is less common than a low-level Gabor filter's edge). A major prediction of our model is that superposition and polysemanticity increase as sparsity increases.
- **Early Transformer MLP neurons are extremely polysemantic.** Our experience is that neurons in the first MLP layer in Transformer language models are often extremely polysemantic. If the goal of the first MLP layer is to distinguish between different interpretations of the same token (eg. "die" in English vs German vs Dutch vs Afrikans), such features would be very sparse and our toy model would predict lots of polysemanticity.

This doesn't mean that everything about our toy model reflects real neural networks. Our intuition is that some of the phenomena we observe (superposition, monosemantic vs polysemantic neurons, perhaps the relationship to adversarial examples) are likely to generalize, while other phenomena (especially the geometry and learning dynamics results) are much more uncertain.

Open Questions

This paper has shown that the superposition hypothesis is true in certain toy models. But if anything, we're left with many more questions about it than we had at the start. In this final section, we review some of the questions which strike us as most important: what do we know, and would we like for future work to clarify?